ELELEM CHILE 2026 · CAPSTONE

Team <name>: <your system>

<scenario A / B / C / D> · <one-line problem>



<team members>

<their organisations>

The problem

The problem in one sentence: who has it, and what the system does about it.

Users: who they are, how many, how they reach the system (web, WhatsApp, internal tool …).

Three real requests a user would make — design for these:
1.
2.
3.

Our decision

Multi-agent	several agents hand off work
Single agent	the model picks tools in a loop
RAG	retrieve, then answer
Pipeline	fixed steps
Prompt	one call, no data

Colour the rung you chose.

The decision: … (prompt / pipeline / RAG / single agent / multi-agent)

Why not one rung simpler? What would break.

Why not one rung fancier? What it would cost.

The failure mode this architecture guards against.

Components and data flow

Diagram: user → … → answer. Boxes for components (LLM calls, tools, data stores, humans), arrows for the flow. A photo of a whiteboard is fine.

One request, step by step:

1.
2.
3.
4.
5.

Concrete choices

	Our choice	Why
Models		
Knowledge and retrieval		
Tools (name → output)		
Memory and state		
Guardrails		

How we will know it works

Test cases (what kind, how many)	Metric	Pass bar

When do you run it? (every prompt change, every model change, nightly …) What would make it pass while the system is wrong?

The numbers and the risks

Cost: tokens per request (in / out) × requests per month × price = $ per month.

Latency: LLM calls and tool calls per request → seconds the user waits.

Top risk	What we do about it
1.	
2.	
3.	